

Introduction to C Programming

C - How to Program, 9th edition

Umer Shahid

Department of Electrical Engineering,
UET Lahore

September 28, 2025

Overview

1. Chapter 2 Objectives
2. Introduction to C Programming
3. Your First C Program
4. Adding Two Integers
5. Memory Concepts
6. Arithmetic in C
7. Decision Making
8. C Keywords
9. Secure Programming
10. Practice Examples

Learning Objectives

By the end of this chapter, you will be able to:

- Write simple C programs
- Use simple input and output statements
- Use fundamental data types
- Learn computer memory concepts
- Use arithmetic operators
- Learn precedence of arithmetic operators
- Write simple decision-making statements
- Begin focusing on secure C programming practices

What is C Programming?

C Language Features

- C facilitates a **structured and disciplined** approach to computer program design
- Developed by Dennis Ritchie at Bell Labs in 1972
- Foundation for many modern programming languages
- Used in system programming, embedded systems, and applications

For Beginners

Think of C as a way to communicate with computers using specific rules and vocabulary, just like learning any new language!

A Simple C Program: Hello World

```
1 // fig02_01.c
2 // A first program in C.
3 #include <stdio.h>
4
5 // function main begins program execution
6 int main(void) {
7     printf("Welcome to C!\n");
8 } // end function main
```

Listing 1: fig02_01.c - Your first C program

Output:

Welcome to C!

Understanding Each Line - Comments

Lines 1-2: Comments

```
// fig02_01.c  
// A first program in C.
```

What are comments?

- Lines that start with `//` are comments
- Comments are for **humans to read**, not the computer
- The computer completely ignores comments
- Used to document what your program does
- Make your code easier to understand

Best Practice

Always write comments to explain what your program does - you'll thank yourself later!

Types of Comments in C

Single-line Comments

```
// This is a single-line comment
```

Multi-line Comments

```
/* This is a multi-line comment  
   It can span multiple lines  
   until we close it */
```

Recommendation

Use `//` for single-line comments to avoid common errors with `/* */` comments

Understanding Each Line - Preprocessor Directive

Line 3: Preprocessor Directive

```
#include <stdio.h>
```

What does this mean?

- Lines starting with # are preprocessor directives
- Processed **before** your program is compiled
- <stdio.h> is a **header file**
- Contains information about input/output functions like printf
- Think of it as "importing a toolbox" with useful functions

Analogy

Like getting a recipe book before cooking - you need the instructions for the ingredients you'll use!

Understanding Each Line - White Space

Line 4: Blank Line

We left this line blank intentionally

White Space in C:

- Blank lines, spaces, and tabs are called **white space**
- Generally ignored by the compiler
- Used to make programs easier to read
- Good programming practice to use white space for organization

Professional Tip

Well-formatted code with proper spacing is easier to read and debug!

Understanding Each Line - The main Function

Line 6: The main Function

```
int main(void) {
```

Breaking it down:

- `main` is a **function** - a building block of C programs
- **Every C program must have a main function**
- Program execution **starts** at `main`
- `int` means `main` returns an integer value
- `void` means `main` doesn't receive any information
- `{` begins the function's body

Key Concept

Think of `main` as the "starting point" of your program - like the front door of a house!

Understanding Each Line - The printf Statement

Line 7: Output Statement

```
printf("Welcome to C!\n");
```

What happens here:

- `printf` is a function that displays text on screen
- Text inside quotes " " is called a **string**
- `\n` is an **escape sequence** for newline
- Every statement must end with semicolon ;
- The semicolon is like a period at the end of a sentence

Remember

Forgetting the semicolon is like forgetting to put a period at the end of a sentence - the computer gets confused!

Escape Sequences

Common Escape Sequences

Sequence	Description
<code>\n</code>	Newline (move to next line)
<code>\t</code>	Tab (horizontal spacing)
<code>\a</code>	Alert (beep sound)
<code>\\</code>	Backslash character
<code>\"</code>	Double quote character

Why Escape Sequences?

Some characters have special meanings in C, so we need special ways to include them in strings.

Understanding Each Line - Closing Brace

Line 8: End of Function

```
} // end function main
```

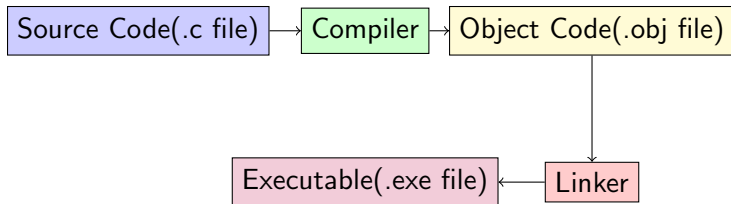
Important points:

- `}` ends the function's body
- Must match with the opening brace `{`
- When program reaches this closing brace, it terminates
- The comment helps identify what we're closing

Brace Matching

Every opening brace `{` must have a matching closing brace `}` - like opening and closing parentheses in math!

Program Compilation and Execution



Steps:

1. Write source code in .c file
2. Compiler converts to object code
3. Linker combines with library functions
4. Creates executable program

Indentation Conventions

Why Indent?

- Makes code easier to read and understand
- Shows the structure of your program
- Professional programming practice
- Helps identify blocks of code

Indentation Rules

- Indent function body one level (recommend 3-4 spaces)
- Be consistent with your spacing
- Use spaces rather than tabs (more predictable)
- Set your code editor to show indentation

Multiple Ways to Use printf

```
1 // Using two printf statements
2 printf("Welcome ");
3 printf("to C!\n");
```

Listing 2: Two printf statements for same output

```
1 // Multiple lines with single printf
2 printf("Welcome\nto\nC!\n");
```

Listing 3: Multiple lines with one printf

Both produce:

```
Welcome
to
C!
```

A More Complex Program: Adding Two Numbers

```
1 // fig02_04.c
2 // Addition program.
3 #include <stdio.h>
4
5 // function main begins program execution
6 int main(void) {
7     int integer1 = 0; // first number
8     int integer2 = 0; // second number
9     printf("Enter first integer: ");
10    scanf("%d", &integer1);
11    printf("Enter second integer: ");
12    scanf("%d", &integer2);
13    int sum = 0;
14    sum = integer1 + integer2;
15    printf("Sum is %d\n", sum);
16 }
```

Listing 4: Addition program

Understanding Variables

What is a Variable?

- A **location in memory** where you can store values
- Think of it as a labeled box where you keep things
- Has a **name**, a **type**, and a **value**
- Must be **defined before use**

Variable Definition Syntax

```
int integer1 = 0;
```

- `int` = data type (whole numbers)
- `integer1` = variable name
- `= 0` = initialization (starting value)
- `;` = statement terminator

Data Types in C

The int Data Type

- Stores whole numbers (integers)
- Examples: 7, -11, 0, 31914
- Cannot store decimal numbers
- Typical range: -32,768 to 32,767 (depends on system)

Why Initialize Variables?

Always initialize variables to avoid using random values that might be left in memory - it's like cleaning a container before putting food in it!

Variable Naming Rules

Valid Identifiers Must:

- Consist of letters, digits, and underscores only
- NOT begin with a digit
- Be case-sensitive (`number` `Number`)
- Not be a C keyword

Examples

Valid: `number1`, `first_name`, `totalScore`

Invalid: `1number`, `first-name`, `int`

Naming Conventions

Camel Case: `totalCommissions`

Underscore: `total_commissions`

The scanf Function

Getting Input from User

```
scanf("%d", &integer1);
```

Breaking it down:

- scanf reads input from keyboard
- "%d" is format control string (expects integer)
- &integer1 tells scanf WHERE to store the value
- & is the **address operator**
- Always use & before variable name in scanf

Critical Warning

Forgetting & before variable name in scanf often causes program crashes!

Format Control Strings

Common Format Specifiers

Specifier	Data Type
%d	Integer (decimal)
%f	Floating-point
%c	Character
%s	String

In printf: %d is a placeholder for a value

In scanf: %d tells what type of input to expect

Memory Aid

Think of %d as a "blank space" that gets filled with a number!

Assignment Statements

Basic Assignment

```
sum = integer1 + integer2;
```

How to read this:

- "sum **gets** the value of integer1 + integer2"
- NOT "sum equals integer1 + integer2"
- The = is the **assignment operator**
- Right side is calculated first, then stored in left side

Important Distinction

= (assignment) is different from == (equality comparison)

Binary Operators

What are Binary Operators?

Operators that work with **two operands** (values)

In the expression: `sum = integer1 + integer2;`

- `+` is a binary operator (operands: `integer1`, `integer2`)
- `=` is a binary operator (operands: `sum`, result of addition)

Formatting Tip

Put spaces around binary operators for better readability:

Good: `sum = integer1 + integer2;`

Poor: `sum=integer1+integer2;`

How Computer Memory Works

Memory Analogy

Think of computer memory like a giant filing cabinet with millions of numbered drawers (addresses), each capable of holding one piece of information.

sum:	117
integer2:	72
integer1:	45

Each variable has:

- A name (integer1, integer2, sum)
- A type (int)
- A value (45, 72, 117)
- A memory location (address)

Destructive vs Non-Destructive Operations

Destructive (Writing to Memory)

When you store a new value in a variable, it **replaces** the old value

- `integer1 = 45; // Old value is lost`
- Like erasing what's written on a whiteboard and writing something new

Non-Destructive (Reading from Memory)

When you read a value from a variable, the original value **remains unchanged**

- `sum = integer1 + integer2; // integer1 and integer2 keep their values`
- Like photocopying a document - original stays intact

Arithmetic Operators

Basic Arithmetic Operators

Operation	Operator	Algebraic	C Expression
Addition	+	$f + 7$	<code>f + 7</code>
Subtraction	-	$p - c$	<code>p - c</code>
Multiplication	*	bm	<code>b * m</code>
Division	/	x/y	<code>x / y</code>
Remainder	%	$r \bmod s$	<code>r % s</code>

Key Points

- Use * for multiplication (not just placing variables together)
- % gives remainder after division (integers only)
- All operators work left-to-right with same precedence

Integer Division and Remainder

Integer Division

When dividing integers, result is also integer (decimal part discarded)

- $7 / 4 = 1$ (not 1.75)
- $17 / 5 = 3$ (not 3.4)

Remainder Operator

Gives the remainder after integer division

- $7 \% 4 = 3$ ($7 \div 4 = 1$ remainder 3)
- $17 \% 5 = 2$ ($17 \div 5 = 3$ remainder 2)
- $8 \% 4 = 0$ ($8 \div 4 = 2$ remainder 0)

Division by Zero

Dividing by zero causes fatal error and program termination!

Operator Precedence Rules

Order of Operations (like PEMDAS in math)

1. **Parentheses** - highest precedence
2. *****, **/**, **%** - left to right
3. **+**, **-** - left to right
4. **=** (assignment) - lowest precedence

Example: `result = 3 + 4 * 5;`

1. First: `4 * 5 = 20`
2. Then: `3 + 20 = 23`
3. Finally: `result = 23`

Precedence Examples

Example 1: Average of 5 numbers

Wrong: `average = a + b + c + d + e / 5;`

This calculates: $a + b + c + d + \frac{e}{5}$

Correct: `average = (a + b + c + d + e) / 5;`

This calculates: $\frac{a+b+c+d+e}{5}$

Example 2: Complex Expression

`z = p * r % q + w / x - y;`

Order: (1) `p * r`, (2) `result % q`, (3) `w / x`, (4) `step2 + step3`, (5) `step4 - y`, (6) assign to `z`

Introduction to Decision Making

What is Decision Making?

- Programs need to make choices based on conditions
- Example: "If grade ≥ 60 , print 'Passed'"
- Uses `if` statements to control program flow

Conditions

- Expressions that evaluate to true or false
- In C: 0 = false, non-zero = true
- Formed using equality and relational operators

Equality and Relational Operators

Comparison Operators

Math	C Op	Example	Meaning
$>$	<code>></code>	<code>x > y</code>	x greater than y
$<$	<code><</code>	<code>x < y</code>	x less than y
$\geq$	<code>>=</code>	<code>x >= y</code>	x greater than or equal to y
$\leq$	<code><=</code>	<code>x <= y</code>	x less than or equal to y
$=$	<code>==</code>	<code>x == y</code>	x equal to y
$\neq$	<code>!=</code>	<code>x != y</code>	x not equal to y

Common Mistake

Don't confuse `=` (assignment) with `==` (equality)!

`=` means "assign", `==` means "compare"

The if Statement

Basic Syntax

```
1 if (condition) {  
2     // statements to execute if condition is true  
3 }
```

Example

```
1 if (number1 == number2) {  
2     printf("%d is equal to %d\n", number1, number2);  
3 }
```

How it works:

- Condition in parentheses is evaluated
- If true (non-zero), statements in braces execute
- If false (zero), statements are skipped

Complete Comparison Example

```
1 printf("Enter two integers: ");
2 scanf("%d %d", &number1, &number2);
3
4 if (number1 == number2) {
5     printf("%d is equal to %d\n", number1, number2);
6 }
7
8 if (number1 != number2) {
9     printf("%d is not equal to %d\n", number1, number2);
10 }
11
12 if (number1 < number2) {
13     printf("%d is less than %d\n", number1, number2);
14 }
15
16 if (number1 > number2) {
17     printf("%d is greater than %d\n", number1, number2);
18 }
```

Listing 5: Comparing two integers

Operator Precedence Summary

Complete Precedence Table (so far)

Operators	Grouping
()	left-to-right
*, /, %	left-to-right
+, -	left-to-right
<, <=, >, >=	left-to-right
==, !=	left-to-right
=	right-to-left

When in Doubt

Use parentheses to make your intentions clear!

C Keywords (Reserved Words)

Keywords You Cannot Use as Variable Names

auto	break	case	char	const
continue	default	do	double	else
enum	extern	float	for	goto
if	int	long	register	return
short	signed	sizeof	static	struct
switch	typedef	union	unsigned	void
volatile	while			

Why Keywords are Reserved

These words have special meanings in C and tell the compiler what to do. Using them as variable names would confuse the compiler!

Introduction to Secure Programming

Why Security Matters

- Poorly written programs can be exploited by attackers
- Security vulnerabilities can lead to data theft or system damage
- Good security practices should be learned from the beginning

What We'll Learn

- Secure use of printf function
- Safe alternatives to potentially dangerous functions
- Following SEI CERT C Coding Standard guidelines

Secure printf Usage

The Problem with Single-Argument printf

Potentially Insecure:

```
1 printf("Welcome to C!\n");
```

Why This Can Be Dangerous

- If the string comes from user input, it might contain format specifiers
- Attackers can craft malicious format strings
- Can lead to reading unauthorized memory locations

The Solution

Use safer alternatives that prevent format string attacks

Secure Alternatives

For Strings with Newline - Use puts()

Instead of: `printf("Welcome to C!\n");`

Use: `puts("Welcome to C!");`

For Strings without Newline - Use %s

Instead of: `printf("Enter age: ");`

Use: `printf("%s", "Enter age: ");`

Why These Are Safer

- puts only displays string + newline, no format processing
- printf with %s explicitly specifies string format
- Both prevent format string attacks

Before and After Examples

Original (Potentially Insecure)

```
1 printf("Hello World!\n");  
2 printf("Enter your name: ");  
3 printf("Welcome to programming!\n");
```

Secure Version

```
1 puts("Hello World!");  
2 printf("%s", "Enter your name: ");  
3 puts("Welcome to programming!");
```

Practice: Calculate Area of a Circle

- Task: Write a program to calculate the area of a circle given its radius.
- Formula: $\text{Area} = \pi r^2$
- Solution:

Code Example

```
1 #include <stdio.h>
2 #define PI 3.14159
3 int main(void) {
4     float radius, area;
5     printf("Enter radius: ");
6     scanf("%f", &radius);
7     area = PI * radius * radius;
8     printf("Area: %.2f\n", area);
9     return 0;
10 }
11
```

Listing 6: Using Escape Sequences

Practice: Calculate Simple Interest

- Task: Write a program to calculate simple interest using the formula: Simple Interest = $\frac{P \cdot R \cdot T}{100}$
- Solution:

Code Example

```
1 #include <stdio.h>
2 int main(void) {
3     float principal, rate, time, interest;
4     printf("Enter Principal, Rate, and Time: ");
5     scanf("%f %f %f", &principal, &rate, &time);
6     interest = (principal * rate * time) / 100;
7     printf("Simple Interest: %.2f\n", interest);
8     return 0;
9 }
10
```

Practice: Temperature Conversion

- Task: Convert temperature from Celsius to Fahrenheit using the formula: $F = \frac{9}{5} \cdot C + 32$
- Solution:

Code Example

```
1 #include <stdio.h>
2 int main(void) {
3     float celsius, fahrenheit;
4     printf("Enter temperature in Celsius: ");
5     scanf("%f", &celsius);
6     fahrenheit = (celsius * 9 / 5) + 32;
7     printf("Temperature in Fahrenheit: %.2f\n", fahrenheit);
8     return 0;
9 }
10
```

Practice: Determine if a Number is Even or Odd

- Task: Write a program to:
 - Take an integer as input.
 - Determine if it is even or odd.
- Solution:

Code Example

```
1 #include <stdio.h>
2 int main(void) {
3     int num;
4     printf("Enter an integer: ");
5     scanf("%d", &num);
6     if (num % 2 == 0) {
7         printf("%d is even\n", num);
8     } else {
9         printf("%d is odd\n", num);
10    }
11    return 0;
12 }
```

Practice: Basic Arithmetic Calculator

- Task: Write a program to perform basic arithmetic operations (+, -, *, /) on two integers based on user choice.
- Solution:

Code Example 1/2

```
1  #include <stdio.h>
2  int main(void) {
3      int num1, num2, choice;
4      printf("Enter two integers: ");
5      scanf("%d %d", &num1, &num2);
6      printf("Choose operation: 1-Add, 2-Subtract, 3-Multiply, 4-Divide: ");
7      scanf("%d", &choice);
8
9      if (choice == 1) {
10         printf("Result: %d\n", num1 + num2);
11     }
```

Practice: Basic Arithmetic Calculator

- Task: Write a program to perform basic arithmetic operations (+, -, *, /) on two integers based on user choice.
- Solution:

Code Example 2/2

```
1     } else if (choice == 2) {  
2         printf("Result: %d\n", num1 - num2);  
3     } else if (choice == 3) {  
4         printf("Result: %d\n", num1 * num2);  
5     } else if (choice == 4 && num2 != 0) {  
6         printf("Result: %d\n", num1 / num2);  
7     } else {  
8         printf("Invalid choice or division by zero.\n");  
9     }  
10    return 0;  
11 }  
12
```

Questions & Discussion

Remember

Programming is a skill developed through practice. Every error you make and fix makes you a better programmer!

Remember: Making mistakes is part of learning.
Every expert programmer started where you are now.